AccessiCheck: Web Accessibility Analyzer

Internship Project Report

Submitted by: Zureen · BCA (Third Year), St. Francis De Sales College, Nagpur

Nov 6, 2026 · @Zureen

1. Introduction

AccessiCheck is a web application that finds accessibility problems in HTML code or on a live web page, explains why they matter in simple words, fixes them automatically and saves a score history.

1.1 Abstract

People who are blind, have low vision or cannot use a mouse depend on screen readers, keyboards and clear page structure. Many websites are built without them in mind, and the developers often do not know. AccessiCheck checks five common problems (missing alt text, unlabeled form fields, empty links, low colour contrast and wrong ARIA roles) and can scan a live public website by its address. Every scan gets a score out of 100, each issue is marked Critical or Medium, and the tool shows the code before and after the fix.

The server is written in Java 17 with Spring Boot, the HTML is read with the Jsoup library, scan history is stored in an H2 database, and the pages use plain HTML, CSS and JavaScript.

1.2 Problem statement

Existing accessibility tools usually give a long technical list of errors. A student or beginner often cannot tell why something is a problem or how to fix it. There is a need for a simple tool that explains each problem in plain language, fixes it with one click and shows progress over time.

1.3 Objectives

- Detect common accessibility problems in HTML code.
- Explain each problem in simple language with a priority level (Critical or Medium).
- Fix the problems automatically and show the code before and after.
- Calculate a score out of 100 and save every scan in a database.
- Scan a live public web page safely from its address.
- Provide a dashboard that is itself easy to use and accessible.

1.4 Scope

AccessiCheck works on pasted HTML code and on public web pages. It is meant for students, beginners and small teams who want to learn and improve accessibility. It checks the six features listed in Chapter 5 and does not replace a full manual accessibility audit.

2. Existing Systems and Comparison

Well-known tools such as WAVE, Google Lighthouse and axe already find accessibility problems, and they cover far more rules than AccessiCheck. Their main strength is wide coverage. AccessiCheck is different in how it teaches and helps a beginner.

Point	Typical existing tools	AccessiCheck
Rules covered	Many WCAG rules	Five rules, plus live URL scan
Explanation	Often technical wording	Short, simple "why it matters" for every issue
Fixing the code	Mostly report the problem and suggest a change	One-click auto-fix of every supported issue
Before / After code	Not the main view	Shown for every issue
Priority	Varies by tool	Critical or Medium, linked to the score
Score history	Usually not kept	Every scan saved in a database, with a progress chart
Setup	Extension or online service	Runs locally with Java and a browser

AccessiCheck is therefore a learning and fixing tool for beginners, not a replacement for a full professional audit.

3. System Requirements and Technologies

AccessiCheck runs on an ordinary laptop with Java 17 or newer and a web browser; the first run needs internet to download libraries through Maven.

3.1 Requirements

Item	Used in this project
Operating system	Windows 10
Java	JDK 21 (project compiled for Java 17)
IDE	Eclipse IDE for Java Developers
Build tool	Maven
Browser	Google Chrome
Internet	Needed for Maven download and URL scan

3.2 Technologies used

Technology	Purpose
Java 17	Main programming language of the server
Spring Boot 3.3	Creates the web server and the REST API; includes the Tomcat server on port 8080
Jsoup 1.18	Reads, searches and edits HTML code
Spring Data JPA	Saves and reads scan history without writing SQL
H2 database	Small file-based database, no installation needed
HTML, CSS, JavaScript	Home page and dashboard (no frameworks)
Maven	Downloads libraries and builds the project

3.3 Why these were chosen

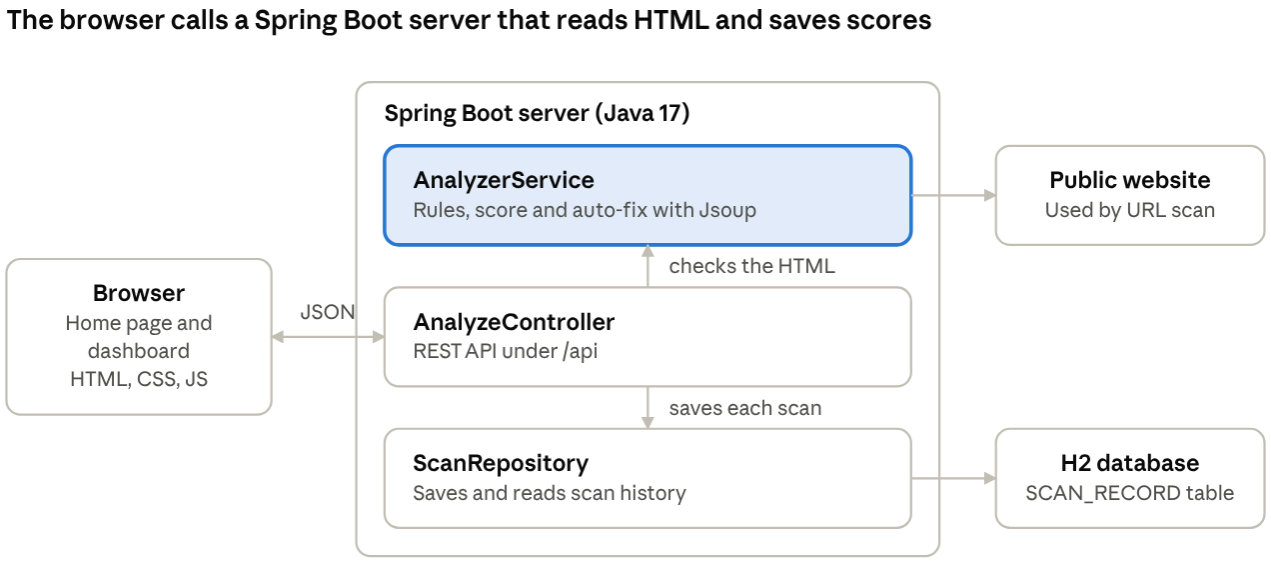
- **Spring Boot** lets a beginner create a working server quickly and is widely used in industry.
- **Jsoup** can parse even untidy HTML and change it, which is needed for auto-fix.
- **H2** keeps the project simple: the whole database is one file and can be viewed in the browser.

4. System Design

AccessiCheck has three layers: the pages in the browser, a Spring Boot server that holds all the accessibility logic, and an H2 database that stores scan history.

4.1 Architecture

The diagram below shows which part calls which.



architecture · browser, server, database

The browser sends JSON to the controller, which asks the service to check the HTML and the repository to save the score.

4.2 Data flow

1. The user pastes HTML code, or types a website address, in the dashboard.
2. The dashboard sends the code or address to the server as JSON (`POST /api/analyze` or `POST /api/analyze-url`).
3. For a website, the server first checks that the address is public, then Jsoup downloads the page.
4. `AnalyzerService` runs the five rule checks. Each problem is recorded and fixed in a working copy of the page.
5. The score is calculated and `AnalyzeController` saves the date, score, issue count and source in the database.
6. The server returns the score, the issues and the fixed HTML. The dashboard draws the gauge and the issue cards.

4.3 Database design

One table, `SCAN_RECORD`, is created automatically by Spring Data JPA.

Column	Type	Meaning
ID	Number (auto)	Unique number of the scan
SCAN_DATE	Text	Date of the scan, for example 6 Nov 2026
SCORE	Number	Accessibility score from 0 to 100
ISSUE_COUNT	Number	Total number of issues found
SCAN_SOURCE	Text	"Pasted HTML" or the website name

The scanned HTML itself is not stored.

4.4 REST API

Method	URL	Input	Output
POST	`/api/analyze`	`{"html": "..."}`	score, issues, total, fixedHtml
POST	`/api/analyze-url`	`{"url": "https://..."}`	same as above, or an error message
GET	`/api/history`	none	list of saved scans

Each issue contains a title, a priority (Critical or Medium), the reason it matters, the code before and the code after the fix.

4.5 Java classes

Class	Job
`AccessiCheckApplication`	Starts the server and adds three sample history rows
`AnalyzeController`	Receives requests, calls the service, saves the scan
`AnalyzerService`	Contains the rules, score and fixing logic
`ColorUtil`	Colour parsing and WCAG contrast maths
`ScanRecord`	The database table as a Java class

Class	Job
`ScanRepository`	Reads and writes scan records

5. Implementation

AccessiCheck parses the page with Jsoup, runs five rules on a working copy, fixes each problem as it finds it, and then calculates a score out of 100.

5.1 The six features

#	Feature	What it finds	Priority	Automatic fix
1	Missing alt text	An `<img>` with no `alt` attribute	Critical	Adds an `alt` placeholder to fill in
2	Unlabeled form field	An `<input>` with no label, `aria-label`, `aria-labelledby` or wrapping `<label>`	Critical	Adds an `id` and a `<label>`
3	Empty link	An `<a>` with no text, no ARIA label and no image	Medium	Adds placeholder link text
4	Colour contrast	Text whose contrast with its background is below 4.5:1 (3:1 for large text)	Critical below 3:1, otherwise Medium	Moves the text colour towards black or white until it passes
5	ARIA roles	A role that does not exist; a custom control (for example `role="button"` on a `<div>`) the keyboard cannot reach; a focusable item with `aria-hidden="true"`	Critical for hidden focusable items, otherwise Medium	Removes the wrong role, adds `tabindex="0"`, or removes `aria-hidden`
6	Live URL scan	Loads a public web page and runs checks 1 to 5 on it	Same as above	Returns the fixed page body

The placeholders ("Describe the image", "Link text") are only a starting point. The developer must replace them with a real description.

5.2 Score

Every Critical issue costs 12 points and every Medium issue costs 7 points. The lowest score is 0.

$$\text{score} = \max\bigl(0,\; 100 - 12 \times \text{Critical} - 7 \times \text{Medium}\bigr)$$

Example: 2 Critical and 1 Medium issue give 100 - 24 - 7 = 69, which the dashboard labels "Needs work". Scores of 80 and above are green, 50 to 79 amber and below 50 red.

5.3 How colour contrast is checked

- The server reads the inline `color` and `background` of each text element and of its parents. If none is set it assumes black text on a white background.
- It calculates the WCAG contrast ratio from the relative luminance of the two colours.

$$\text{ratio} = \dfrac{L_{\text{lighter}} + 0.05}{L_{\text{darker}} + 0.05}$$

- Normal text needs at least 4.5:1. Large text (24 px, or 18.66 px and bold) needs 3:1.
- For the fix, the text colour is moved step by step towards black or white, whichever contrasts more, until the ratio passes.

5.4 Auto-fix

Because Jsoup edits the parsed copy while the rules run, the server can return the corrected HTML (`fixedHtml`) with every response. When the user presses **Apply all fixes and rescan**, the dashboard puts that code in the box and scans it again, which normally gives 100.

5.5 Safety of the URL scan

- Only `http://` and `https://` addresses are accepted.
- Local and private addresses (localhost, 127.x, 10.x, 192.168.x, 172.16 to 172.31 and link-local) are blocked. This prevents server-side request forgery (SSRF), where a web tool is tricked into reading a private network.
- The download times out after 10 seconds, and at most 200 issues are returned for one page.
- The scanned HTML is not saved; only date, score, issue count and source are stored.

5.6 Home page and dashboard

The **home page** (`index.html`) introduces the tool with a hero section, a live count of scans from the database, a "Try it" box, a screen reader demonstration with sound, a FAQ and scroll animations. The **dashboard** (`dashboard.html`) has the New Scan screen (paste HTML or scan a URL), an animated score gauge, confetti at 100, issue cards with Before and After code, copy and download buttons, a scan history table, a score chart, dark mode and large text mode.

The pages follow the accessibility rules the tool teaches: every field has a label, buttons can be used with the keyboard, results are announced to screen readers, and animations stop when the user has asked the system for reduced motion.

6. Testing and Results

All ten test cases passed on the final version, and every scan after "Apply all fixes" reached 100.

6.1 Test cases

#	Input	Expected result	Result
1	A good page: image with alt text, labeled input, link with text	Score 100, no issues	Pass
2	Broken sample: image without alt, unlabeled input, empty link	Score 69 (2 Critical, 1 Medium)	Pass
3	Apply all fixes and rescan after test 2	Score 100	Pass
4	Three images: one without alt, one with `alt=""` , one with `alt="Logo"`	Score 88 (1 Critical)	Pass
5	Form with one unlabeled input and two empty links	Score 74 (1 Critical, 2 Medium)	Pass
6	`<p style="color:#aaa">Light grey</p>`	Score 88 (1 Critical, low contrast)	Pass
7	`role="buttn"` , a `<div role="button">` , and a link with `aria-hidden="true"`	Score 74 (1 Critical, 2 Medium)	Pass
8	URL scan of `https://example.com`	Result shown, source saved as example.com	Pass
9	URL scan of `localhost:8080`	Blocked with a safety message	Pass

#	Input	Expected result	Result
10	Stop and restart the server	Scan history is still there	Pass

Tests 4 to 7 matter because they show the same score can come from different rules, and that `alt=""` (a decorative image) is correctly not reported.

6.2 Screenshots

The screenshots below show the running application. Dark mode and light mode are both included.

7. Limitations, Future Scope and Conclusion

AccessiCheck covers five rules and a URL scan, so it is a good first step for beginners, but it does not find every accessibility problem.

7.1 Limitations

- Colour contrast is checked for inline styles only, not for external or internal CSS.
- The URL scan reads the HTML sent by the server. Content that JavaScript adds later is not seen.
- Pages that need a login, or that block automated requests, cannot be scanned.
- Only the code structure is checked. A person must still judge whether an alt text or label really makes sense.
- There are no user accounts, so all scans are in one shared history.

7.2 Future scope

- More rules: heading order, page language, page title, tables and video captions.
- Contrast check for CSS files and for text on images.
- Export of the history and reports as PDF.
- User accounts and separate project folders.
- A browser extension that scans the page you are viewing.

7.3 Conclusion

AccessiCheck meets its objectives: it finds common accessibility problems, explains them in simple words, fixes them in one click, scans live websites safely and keeps a history of scores in a database. The project gave practical experience with Java, Spring

Boot, a REST API, Jsoup, JPA with an H2 database, and with building a front end that is itself accessible. The design is easy to extend, because every new rule is one method in `AnalyzerService`.

7.4 References

1. W3C. Web Content Accessibility Guidelines (WCAG) 2.1. w3.org/TR/WCAG21
2. W3C. Accessible Rich Internet Applications (WAI-ARIA) 1.2. w3.org/TR/wai-aria-1.2
3. Spring Boot reference documentation. spring.io/projects/spring-boot
4. Jsoup: Java HTML parser. jsoup.org
5. H2 Database Engine. h2database.com
6. WebAIM. WAVE web accessibility evaluation tool. wave.webaim.org
7. OWASP. Server Side Request Forgery. owasp.org